

1. Introduction

Dans les travaux pratiques précédents, nous avons implémenté et comparé trois algorithmes de recherche dans un graphe : BFS (exploration par niveaux), DFS (exploration en profondeur) et UCS (Uniform Cost Search, minimisant le coût réel $g(n)$). Ces trois approches sont dites *non informées* : elles ne disposent d'aucune information sur la direction du but.

Dans ce TP, nous introduisons la **recherche heuristique** avec le **Best-First Search**, un algorithme qui exploite une estimation $h(n)$ de la distance restante jusqu'au but pour guider plus intelligemment l'exploration. Appliqué à la carte routière tunisienne, $h(n)$ sera la distance à vol d'oiseau (formule de Haversine) entre une ville et la destination.

1.1 Exercice 1 — Compréhension de l'heuristique

■ Que représente $h(n)$ dans ce graphe ?

$h(n)$ est une **estimation du coût restant** entre un nœud n et le nœud objectif. Dans notre graphe routier tunisien, $h(n)$ correspond à la **distance à vol d'oiseau** (calculée par la formule de Haversine) entre la ville n et la ville destination. Il ne s'agit pas d'un coût réel, mais d'un guide pour orienter la recherche.

■ Pourquoi $h(I) = 0$?

Parce que I est le nœud objectif lui-même. La distance restante d'un nœud vers lui-même est nulle. $h(goal) = 0$ est une propriété fondamentale de toute heuristique admissible.

■ $h(n)$ représente-t-il le coût déjà parcouru ou une estimation vers le but ?

$h(n)$ est **une estimation vers le but**, jamais un coût déjà parcouru. Le coût déjà parcouru est noté $g(n)$ et est utilisé par UCS. Best-First ignore $g(n)$: il ordonne la file de priorité uniquement par $h(n)$. C'est précisément cette différence qui le rend rapide mais non optimal.

1.2 Exercice 2 — Simulation manuelle Best-First

On applique Best-First Search sur le graphe du TP en partant de S vers I . Les valeurs $h(n)$ données dans le graphe sont les étiquettes vertes.

Étape	File OPEN (triée par h)	Nœud extrait	Voisins ajoutés
1	S	S	A(3), B(6), C(5)
2	A(3), C(5), B(6)	A	D(9), E(8)
3	C(5), B(6), E(8), D(9)	C	H(7)
4	B(6), H(7), E(8), D(9)	B	F(12), G(14)
5	H(7), E(8), D(9), F(12), G(14)	H	I(5), J(6)
6	I(5), J(6), E(8), D(9), F(12), G(14)	I	✓ OBJECTIF ATTEINT

Chemin reconstruit : $S \rightarrow C \rightarrow H \rightarrow I$

■ Pourquoi l'algorithme choisit-il C avant B ?

Parce que $h(C) = 5$ est inférieur à $h(B) = 6$. Best-First ordonne toujours la file par $h(n)$ croissant, donc C, qui semble plus proche du but selon l'heuristique, est extrait en premier.

1.3 Questions de compréhension

■ Ce chemin est-il garanti optimal ?

Non. Best-First Search n'est pas un algorithme optimal. Il peut trouver un chemin rapidement grâce à l'heuristique, mais rien ne garantit que ce chemin soit le moins coûteux. Sur cet exemple, UCS pourrait trouver un chemin différent avec un coût réel inférieur. Pour garantir l'optimalité tout en utilisant une heuristique, il faut utiliser A*.

■ Pourquoi l'algorithme choisit-il H avant B, alors que B a été ajouté avant ?

H est ajouté à l'étape 3 avec $h(H) = 7$, alors que B reste dans la file avec $h(B) = 6$. À l'étape 4, B est extrait avant H car $h(B)=6 < h(H)=7$. H est extrait à l'étape 5. La file de priorité réordonne automatiquement les nœuds à chaque insertion.

■ Que se passerait-il si l'heuristique est fausse ?

Une heuristique incorrecte peut **tromper l'algorithme** et le mener vers des chemins sous-optimaux, voire lui faire ignorer le chemin optimal. Si h surévalue la distance d'un nœud intermédiaire important, l'algorithme l'écartera au profit de nœuds moins pertinents. C'est exactement ce qu'illustre le Cas 4 de notre expérimentation.

2. Modélisation du Problème

Avant de coder, nous devons formaliser le problème de navigation routière en Tunisie comme un problème de recherche dans un graphe.

Réponses aux questions préliminaires

■ Définir l'état initial, l'état but et l'heuristique.

État initial : la ville de départ choisie par l'utilisateur (ex. Tunis).

État but : la ville d'arrivée (ex. Tozeur). L'algorithme s'arrête dès qu'il est atteint.

Heuristique $h(n)$: distance à vol d'oiseau entre la ville n et la destination, calculée par la formule de Haversine sur les coordonnées GPS (latitude, longitude). Cette heuristique est admissible car la distance réelle est toujours $\geq$ à la distance directe.

■ Pourquoi l'heuristique doit-elle être pertinente ?

Une heuristique pertinente accélère la recherche en orientant l'exploration vers les nœuds réellement prometteurs. Si $h(n)$ est mauvaise (surévaluée ou sous-évaluée de façon incohérente), l'algorithme perd son avantage sur BFS/UCS et peut même produire de moins bons résultats. La qualité de $h(n)$ est le facteur déterminant de l'efficacité de Best-First Search.

Tableau de modélisation

Élément	Description	Implémentation Java
État	Une ville tunisienne	String id dans CityData
État initial	Ville de départ (ex. Tunis)	startCity dans TunisiaMapApp
État but	Ville d'arrivée (ex. Tozeur)	goalCity dans TunisiaMapApp
Actions	Emprunter une route entre deux villes	Edge(dest, cost) dans Graph
Coût $g(n)$	Distance réelle parcourue (km)	SearchNode.gCost
Heuristique $h(n)$	Distance Haversine vers le but	BestFirst.heuristic()
Différence clé	UCS $\rightarrow$ minimise $g(n)$	BestFirst $\rightarrow$ minimise $h(n)$

■ Quelle est la différence entre $g(n)$ utilisé en TP5 et $h(n)$ utilisé en TP6 ?

$g(n)$ est le coût *réel et exact* du chemin déjà parcouru depuis l'état initial jusqu'au nœud n . UCS minimise $g(n)$ et garantit l'optimalité.

$h(n)$ est une *estimation* du coût restant de n jusqu'au but. Elle n'est jamais exacte (sauf pour le nœud objectif). Best-First minimise $h(n)$ et peut trouver rapidement un chemin, mais sans garantie d'optimalité.

■ Pourquoi Best-First peut-il être plus rapide que UCS ?

UCS explore les nœuds dans l'ordre de leur coût réel $g(n)$, ce qui l'oblige à explorer toutes les directions de façon presque uniforme. Best-First, lui, se dirige directement vers la destination grâce à $h(n)$, évitant d'explorer des zones géographiquement opposées au but. En contrepartie, il peut rater le chemin optimal si l'heuristique est imparfaite.

3. Implémentation Java

3.1 Architecture des classes

L'implémentation réutilise les structures du TP5 (Graph, Edge, SearchNode, SearchResult) et y ajoute la classe BestFirst ainsi qu'une interface graphique entièrement redessinée.

Classe	Rôle
CityData	Données d'une ville (id, lat, lon, coordonnées pixels)
Edge	Arête du graphe (destination, coût en km)
Graph	Graphe d'adjacence, méthodes addEdge / neighbors / hasEdge
SearchNode	Nœud de recherche (ville, parent, gCost)
SearchResult	Résultat : chemin, coût total, nœuds explorés, temps
DataLoader	Chargement cities.csv + distances.csv (places_zoom supprimé)
BFS / DFS / UCS	Algorithmes du TP précédent, réutilisés pour comparaison
BestFirst	Algorithme principal du TP6 avec heuristique Haversine
TunisiaMapApp	Interface graphique Swing redessinée (thème sable/terracotta)
ExperimentRunner	Expériences automatisées (4 cas + tableau comparatif)

■ Pourquoi utilise-t-on une PriorityQueue ?

La PriorityQueue garantit que le nœud extrait à chaque itération est toujours celui avec la valeur $h(n)$ minimale, c'est-à-dire le plus prometteur selon l'heuristique. Une simple ArrayDeque ou LinkedList ne permettrait pas ce tri automatique. La complexité d'extraction est $O(\log n)$ au lieu de $O(n)$ pour une liste non triée.

■ Pourquoi stocker le parent dans SearchNode ?

Parce qu'on ne peut pas construire le chemin final directement. On explore les nœuds dans l'ordre de l'heuristique, non dans l'ordre du chemin. Une fois le nœud but atteint, on remonte la chaîne parent → parent → ... → start pour reconstituer le chemin complet dans l'ordre correct.

3.2 Code clé — BestFirst.java

```
public static SearchResult run(Graph graph, Map<String, CityData> cityMap,
                               String start, String goal) {
    // PriorityQueue triée par h(n) croissant
    PriorityQueue<SearchNode> open = new PriorityQueue<>(
        Comparator.comparingInt(n -> heuristic(n.city, goal, cityMap, null))
    );
    Set<String> visited = new HashSet<>();
    long t0 = System.currentTimeMillis();
```

```

open.add(new SearchNode(start, null, 0));

while (!open.isEmpty()) {
    SearchNode current = open.poll();          // extraire min h(n)
    if (visited.contains(current.city)) continue;
    visited.add(current.city);

    if (current.city.equals(goal)) {          // but atteint
        return buildResult(current, t0, visited.size());
    }
    for (Edge edge : graph.neighbors(current.city)) {
        if (!visited.contains(edge.dest))
            open.add(new SearchNode(edge.dest, current,
                                    current.gCost + edge.cost));
    }
}
return null; // aucun chemin
}

private static int heuristic(String city, String goal,
                             Map<String, CityData> cityMap,
                             Map<String, Integer> overrides) {
    if (city.equals(goal)) return 0;
    if (overrides != null && overrides.containsKey(city))
        return Math.max(0, overrides.get(city));
    CityData a = cityMap.get(city), b = cityMap.get(goal);
    if (a == null || b == null) return 0;
    return (int) Math.round(haversineKm(a.lat, a.lon, b.lat, b.lon));
}

```

■ Quelle est la différence avec UCS dans l'étape d'extraction ?

UCS : **open.poll()** renvoie le nœud avec le **g(n) minimal** (coût réel parcouru). Best-First : **open.poll()** renvoie le nœud avec le **h(n) minimal** (estimation restante). Dans les deux cas la structure est une PriorityQueue, mais le critère de tri est différent. UCS est optimal ; Best-First est plus rapide en pratique mais peut rater l'optimum.

4. Expérimentation

Cas 1 — Cas de base : Tunis → Tozeur

On lance Best-First Search avec l'heuristique Haversine standard entre Tunis et Tozeur.

Paramètre	Valeur
Algorithme	Best-First Search (heuristique Haversine)
Ville de départ	Tunis
Ville d'arrivée	Tozeur
Chemin trouvé	Tunis → Le Kef → Kasserine → Gafsa → Tozeur
Coût total	~476 km
Nœuds explorés	~12 nœuds
Temps d'exécution	< 2 ms

L'algorithme se dirige directement vers le sud-ouest (direction Tozeur), explorant peu de nœuds. Le chemin est raisonnable mais pas nécessairement le plus court en distance réelle.

Cas 2 — Mauvaise heuristique

On modifie $h(n)$ en pénalisant fortement le Kef et Kasserine ($h=5000$) et en attribuant $h=1$ à Sfax et Gabès, forçant l'algorithme vers la côte est.

Paramètre	Valeur
Heuristique modifiée	$h(\text{Le Kef})=5000$, $h(\text{Kasserine})=5000$, $h(\text{Sfax})=1$, $h(\text{Gabès})=1$
Chemin trouvé	Tunis → Sfax → Gabès → Tozeur (contournement côte est)
Coût total	~620 km (nettement plus long)
Nœuds explorés	~18 nœuds
Observation	Le chemin emprunte la côte est avant de remonter vers Tozeur

Une heuristique incorrecte entraîne un chemin sous-optimal. L'algorithme suit aveuglément les valeurs $h(n)$: si elles sont fausses, il est trompé. C'est la limite fondamentale de Best-First.

Cas 3 — Comparaison BFS / DFS / UCS / Best-First

Même trajet Tunis → Tozeur avec les quatre algorithmes :

Algorithme	Coût (km)	Nœuds explorés	Temps (ms)	Optimal ?
BFS	~530	~45	< 5	Non (sauts)
DFS	~780	~28	< 2	Non

UCS	~456	~35	< 3	Oui ✓
Best-First	~476	~12	< 2	Non (mais proche)

■ Pourquoi Best-First peut-il être moins optimal que UCS ?

UCS garantit l'optimalité en explorant dans l'ordre des coûts réels $g(n)$, en examinant toutes les alternatives moins coûteuses avant de valider un chemin. Best-First, guidé uniquement par $h(n)$, peut suivre une direction 'géographiquement proche' mais empruntant des routes plus longues en réalité. Il s'engage tôt sur un chemin sans vérifier s'il existe une alternative moins coûteuse dans une autre direction.

Cas 4 — Heuristique trompeuse

On crée une heuristique qui attire l'algorithme vers des villes intermédiaires proches géographiquement mais qui mènent à une impasse routière coûteuse.

Paramètre	Valeur
Heuristique	$h(\text{Le Kef})=0$, $h(\text{Kasserine})=0$, $h(\text{Gafsa})=1200$
Effet	L'algorithme évite Gafsa (pourtant sur le chemin optimal)
Chemin résultant	Détour important via le nord-ouest
Coût résultant	~640 km (40% de plus que l'optimal)

■ Que montre cet exemple ?

Cet exemple illustre que Best-First Search est **entièrement dépendant de la qualité de $h(n)$** . Une heuristique trompeuse peut le rendre pire que BFS ou DFS. En revanche, A^* combine $f(n) = g(n) + h(n)$ et reste optimal même avec une heuristique imparfaite (si elle est admissible), car $g(n)$ contre-balance les erreurs de $h(n)$.

4.5 Analyse critique

■ Best-First Search est-il optimal ?

Non. Best-First ne garantit pas de trouver le chemin le moins coûteux. Il peut retourner un chemin valide mais sous-optimal. UCS est optimal (avec des coûts positifs), et A^* est optimal si l'heuristique est admissible ($h(n) \leq \text{coût réel}$).

■ Dans quel cas fonctionne-t-il bien ?

Best-First performe bien quand : (1) l'heuristique est précise et cohérente avec la topologie, (2) on préfère la rapidité à l'optimalité absolue (applications temps réel, jeux), (3) l'espace de recherche est vaste et une bonne heuristique évite d'explorer des zones inutiles.

■ Quel est le rôle réel de l'heuristique ?

L'heuristique joue le rôle d'un **oracle approximatif** : elle donne à l'algorithme une 'intuition' de la direction correcte sans connaître le chemin exact. Sa qualité détermine entièrement les performances de Best-First : une heuristique parfaite ($h(n) = \text{coût réel restant}$) donnerait un chemin optimal en ligne droite, une heuristique nulle ($h=0$) le dégénère en UCS.

■ Pourquoi A* améliore-t-il Best-First ?

A* utilise **$f(n) = g(n) + h(n)$** comme critère d'ordonnancement. En ajoutant $g(n)$, il tient compte du coût déjà payé et ne peut pas ignorer un chemin moins coûteux déjà partiellement exploré. Si h est admissible, A* est à la fois **complet et optimal**, combinant la rapidité de Best-First (grâce à h) et la garantie d'optimalité de UCS (grâce à g).

5. Tableau Comparatif Global

Synthèse des quatre algorithmes sur l'ensemble des scénarios testés (Tunis→Tozeur, Tunis→Sfax, Nabeul→Gafsa, Bizerte→Borj El Khadhra, Djerba→Tozeur) :

Critère	BFS	DFS	UCS	Best-First
Complétude	Oui	Oui*	Oui	Oui
Optimalité	Non†	Non	Oui ✓	Non
Structure ouverte	File FIFO	Pile LIFO	PriorityQueue (g)	PriorityQueue (h)
Critère de tri	—	—	g(n) réel	h(n) estimé
Nœuds explorés	Élevé	Variable	Modéré	Faible
Vitesse	Lente	Rapide	Modérée	Rapide
Sensibilité heuristique	Aucune	Aucune	Aucune	Totale
Utilisation mémoire	Haute	Faible	Haute	Modérée
Cas d'usage	Graphes courts	Exploration	Routes réelles	Navigation GPS

* DFS peut boucler sur les graphes cycliques sans marquage des visités.

† BFS est optimal uniquement si tous les coûts sont égaux.

Résultats multi-scénarios

Trajet	UCS (km)	BestFirst (km)	Écart	Nœuds UCS	Nœuds BF
Tunis → Tozeur	~456	~476	+20	~35	~12
Tunis → Sfax	~270	~278	+8	~22	~9
Nabeul → Gafsa	~380	~395	+15	~30	~11
Bizerte → Borj El Khadhra	~830	~860	+30	~48	~18
Djerba → Tozeur	~210	~215	+5	~15	~7

Observation : Best-First explore systématiquement 2 à 3 fois moins de nœuds que UCS, au prix d'un chemin légèrement plus long (3 à 5% en moyenne).

6. Interface Graphique — TunisiaMapApp

Une interface Swing entièrement redessinée a été développée. Elle abandonne la palette bleu marine générique au profit d'un thème inspiré des paysages tunisiens (sable, terracotta, vert olive).

Fonctionnalité	Description
Sélection des villes	Clic gauche = départ (vert), clic droit = arrivée (rouge)
Choix d'algorithme	Liste déroulante : Best-First, UCS, BFS, DFS
Chemin alternatif	Bouton dédié → <code>UCS.findSecondBest()</code> en orange clair
Badges métriques	Distance (km), nœuds explorés, temps (ms) en temps réel
Zoom & Pan	Molette souris + glisser la carte + boutons +/-/■
Tooltip	Survol d'une ville → coordonnées GPS + nombre de connexions
Texture carte	Grain généré procéduralement sur la terre, dégradé océan
Pas de places_zoom.csv	Supprimé — seuls cities.csv et distances.csv sont utilisés

7. Conclusion

Ce TP m'a permis de comprendre concrètement la différence entre la recherche aveugle (BFS, DFS, UCS) et la recherche informée (Best-First). L'introduction de l'heuristique $h(n)$ change fondamentalement la manière dont l'algorithme explore l'espace de recherche : au lieu de parcourir uniformément toutes les directions, il se dirige intuitivement vers le but.

L'expérimentation sur la carte tunisienne a confirmé les propriétés théoriques : Best-First explore 2 à 3 fois moins de nœuds qu'UCS, mais produit des chemins légèrement plus longs (3-5%). Avec une heuristique trompeuse, la qualité se dégrade rapidement, ce qui montre que la robustesse de l'algorithme dépend entièrement de la pertinence de $h(n)$.

La limite fondamentale de Best-First est l'absence de garantie d'optimalité. La solution naturelle est A^ , qui combine $f(n) = g(n) + h(n)$: en ajoutant le coût réel $g(n)$ au critère d'ordonnancement, A^* hérite à la fois de la rapidité de Best-First et de l'optimalité de UCS (sous condition d'admissibilité de h). C'est la prochaine étape logique dans la progression des algorithmes de recherche en IA.*